

Prototype Ingestion Pipeline -- Technical Spec (v2)

This document specifies how the Prototype Ingestion Pipeline works as implemented. It supersedes the [v1 technical spec](#), which was written before implementation began. Changes from v1 are noted inline where significant. The companion [vision document](#) and [task decomposition](#) preceded this work.

1. Prototype Convention

There is no separate folder or metadata file. Prototypes live in `src/pages/` in the `lista-payroll-design` repo, exactly where designers build them.

Designers work on prototype branches -- each branch representing a screen or feature in progress. They iterate freely on these branches without triggering the pipeline. When a prototype is considered ready to land in production, the designer merges their branch to main.

Merging to main means the prototype is ready for ingestion. The pipeline watches for changes to page files on main and treats every changed page as a prototype to process.

```
None
lista-payroll-design/
  src/
    pages/
      payroll-setup/          <-- pipeline detects changes here
        PayrollSetupPage.tsx
      steps/
        CompanyProfileStep.tsx
        ...
      DashboardPage.tsx      <-- pipeline detects changes here
      PayrollPage.tsx
```

A prototype's identity is derived from its location in `src/pages/`:

- **Single-file pages** (e.g., `DashboardPage.tsx`): the PascalCase file name (minus extension) is converted to kebab-case to produce the prototype slug (e.g., `dashboard-page`).
- **Folder-based pages** (e.g., `payroll-setup/`): the folder name is the prototype slug directly.

The pipeline derives the prototype's name, route, and source files automatically (see sections 3-5). Designers have zero extra steps beyond their normal workflow of building and merging.

Slug Exclusion

Slugs can be excluded from ingestion via `.github/config/prototype-ingestion.json`:

```
JSON
{
  "exclude": ["not-found-page"]
}
```

The `exclude` array accepts exact slugs and wildcard patterns (e.g., `"reports-*"`). Excluded slugs are skipped entirely and logged as warnings. A missing or malformed config file is treated as an empty exclude list -- the pipeline never fails due to config issues.

2. Change Detection

The pipeline determines which prototypes to ingest by examining git changes on merge to main.

Mechanism

1. On push to main, the GitHub Action computes the diff: `git diff --name-only HEAD~1 HEAD -- src/pages/`
2. Changed `.tsx` and `.ts` files are grouped by their prototype slug:
 - Files directly in `src/pages/` are grouped by file name converted to kebab-case (e.g., `DashboardPage.tsx` -> slug `dashboard-page`)
 - Files inside subdirectories are grouped by their first-level subdirectory under `src/pages/` (e.g., `src/pages/payroll-setup/steps/CompanyProfileStep.tsx` -> slug `payroll-setup`)
3. Slugs matching any pattern in the `exclude` config are removed.
4. Each remaining unique slug becomes a prototype to process.

The implementation lives in `.github/scripts/detect-changes.js`, which writes `has_changes` (`"true"` or `"false"`) and `slugs` (JSON array) to `GITHUB_OUTPUT`.

Manual Trigger

v2 addition: The workflow supports `workflow_dispatch` with an optional `slugs` input -- a comma-separated list of prototype slugs to process (e.g., `employees, payroll-onboarding`). When provided, change detection is skipped and the given slugs are processed directly. When empty, the script falls back to auto-detection from the last commit.

Rules

- **Added or modified `.tsx/.ts` files** in `src/pages/` trigger ingestion for the corresponding prototype.
 - **Deleted files** are ignored. The pipeline does not take action on removal.
 - **Non-page files** (e.g., shared components in `src/components/`, context files) are ignored by the path filter.
-

3. Route Resolution

Since there is no metadata file, the pipeline derives each prototype's route from `App.tsx`.

Mechanism

1. The script `.github/scripts/resolve-routes.js` parses `src/App.tsx`.
2. It reads `import` statements that pull from `./pages/...` and matches them against `<Route>` elements to build a slug-to-route mapping.
3. A slug may map to **one or more routes** (e.g., a page with nested routes). The output is a JSON object where values can be strings or arrays.
4. If no route is found for a slug, screenshot capture is skipped but metadata extraction still proceeds.

Output

```
JSON
{
  "dashboard-page": "/dashboard",
  "payroll-onboarding": "/payroll/setup"
}
```

Example

The pipeline detects changes in `src/pages/payroll-setup/`. It parses `App.tsx`, finds `<Route path="payroll/setup" element={<PayrollSetupPage />} />`, and resolves the route to `/payroll/setup`.

4. Screenshot Capture

The pipeline captures visual snapshots of each prototype using an **agentic workflow**. A Copilot agent with access to a Playwright MCP server navigates the running application, interacts with the UI, and captures screenshots -- including multi-step flows like wizards where individual steps are not directly addressable by URL.

Process

1. The GitHub Action installs dependencies (`npm ci`) and starts the Vite dev server (`npm run dev &`) in the background.
2. The action health-checks the dev server by polling `http://localhost:5173` for up to 30 seconds. If the server fails to respond, the workflow exits with an error.
3. Playwright Chromium is installed (`npx playwright install --with-deps chromium`), and `@playwright/mcp` and `@github/copilot` are installed globally.
4. The script `.github/scripts/capture-screenshots.js` generates a prompt per slug from the resolved routes and the prompt template at `.github/prompts/capture-screenshot.prompt.md`.
5. For each prompt, the workflow invokes the **Copilot CLI** with the Playwright MCP server attached:

Shell

```
copilot --allow-all-tools \
  --additional-mcp-config
'{"mcpServers":{"playwright":{"command":"npx","args":["@playwright/mcp@latest",
"--headless"]}}}' \
  -p "$PROMPT" -s --no-ask-user
```

6. The agent navigates to each route and captures a full-page screenshot at a fixed viewport (1440x900). Screenshots are saved as `screenshots/{slug}.png`, with additional routes producing `screenshots/{slug}-2.png`, etc.

7. For multi-step flows (wizards, onboarding flows, tabbed interfaces), the agent advances through steps and captures `screenshots/{slug}-step-1.png`, `screenshots/{slug}-step-2.png`, etc.
8. If a route returns a 404 or screenshot capture fails for a slug, a warning is logged and the pipeline continues.
9. All screenshots are uploaded as a GitHub Actions artifact (`prototype-screenshots`, 30-day retention).
10. The Vite dev server and Playwright MCP server are started once per workflow run and reused across all prototypes.

Multi-Step Navigation

The agent handles screens that are not directly addressable by URL:

- **Wizards:** The agent clicks through "Next" / "Continue" buttons, capturing each step. It uses visible step indicators or progress rails to confirm it has visited all steps.
- **Tabbed interfaces:** The agent clicks each tab and captures the resulting view.
- **Nested views:** The agent identifies expandable sections, accordions, or drill-down links and captures their expanded state.

For routes with dynamic parameters (like `:id` or `:token`), the agent skips them or uses a reasonable sample value.

Dependencies

- `@playwright/mcp` for browser automation via MCP tools (installed at workflow runtime, not in `package.json`)
- `@github/copilot` CLI for AI agent invocation (installed at workflow runtime)
- Chromium browser installed in the GitHub Action (`npm install --with-deps chromium`)
- `COPILOT_GITHUB_TOKEN` secret (uses `${{ secrets.COPILOT_GITHUB_TOKEN }}` in the workflow, sourced from `GITHUB_TOKEN`)
- The Vite dev server must be able to serve all routes defined in `App.tsx`

v2 change from v1: The v1 spec described a generic "AI agent" and mentioned API credentials for an unspecified agent provider. The implementation uses **GitHub Copilot CLI** (`@github/copilot`) as the agent, authenticated via `GITHUB_TOKEN`. The Playwright MCP server is attached as additional MCP config to Copilot rather than being managed separately.

5. Metadata Extraction

The pipeline automatically extracts all technical metadata from the prototype's source files. There is no authored metadata -- everything is derived from the code. A **separate Copilot agent invocation** (distinct from the screenshot capture agent) reads the source files and produces structured JSON output.

v2 change from v1: The v1 spec described a single agent performing both screenshot capture and metadata extraction. The implementation uses **separate Copilot invocations** for each concern -- the screenshot agent uses Playwright MCP tools, while the metadata agent reads source files directly. This separation allows independent retry logic for metadata extraction.

Process

1. `.github/scripts/extract-metadata.js` generates prompts from the template at `.github/prompts/extract-metadata.prompt.md`, injecting the slug, human-readable name, and source file paths.
2. `.github/scripts/run-metadata-extraction.js` orchestrates the Copilot CLI invocations with validation and retry:
 - Calls the Copilot agent with each prompt
 - Validates the output using `.github/scripts/validate-metadata.js`
 - On validation failure, builds a retry prompt with error feedback and re-invokes
 - Retries up to `METADATA_MAX_RETRIES` times (default: 2)
 - Writes validated JSON to the `metadata/` directory as `metadata/{slug}.json`

Validation

v2 addition: Metadata validation was not described in the v1 spec. The implementation includes a robust validation pipeline.

The `validate-metadata.js` script performs three stages:

1. **Sanitize** -- strips markdown code fences and surrounding prose from the agent output
2. **Parse** -- attempts `JSON.parse` on the sanitized text
3. **Validate** -- checks the parsed object against a Zod schema

The Zod schema (`MetadataSchema`) enforces:

```
JavaScript
{
```

```

    name: z.string().min(1),
    slug: z.string().min(1),
    components: z.array(ComponentSchema).min(1), // at least one component
    interactiveElements: z.array(InteractiveElementSchema).default([])
  }

```

Where `ComponentSchema` is recursive:

```

JavaScript
{
  name: z.string().min(1),
  props: z.record(z.string(), z.unknown()).optional().default({}),
  children: z.array(ComponentSchema).optional().default([])
}

```

And `InteractiveElementSchema`:

```

JavaScript
{
  type: z.string().min(1),
  label: z.string().min(1),
  position: z.string().optional().default("")
}

```

On validation failure, the retry prompt includes the specific Zod errors, instructing the agent to fix its output.

Prototype Name

Derived from the prototype slug, converted from kebab-case to title case (e.g., `payroll-setup` becomes "Payroll Setup"). For single-file pages, the `Page` suffix is stripped (e.g., `DashboardPage.tsx` becomes "Dashboard").

Component Hierarchy

The agent reads the prototype's source files and produces a **component hierarchy** as a recursive JSON tree showing how Modus Web Components are composed together. The agent follows imports across files to build the full tree, enriched with prop values like labels, placeholder text, and button content.

Example output for a payroll setup wizard:

JSON

```
{
  "name": "PayrollSetupPage",
  "props": {},
  "children": [
    {
      "name": "ModusStepper",
      "props": {},
      "children": [
        {
          "name": "CompanyProfileStep",
          "props": {},
          "children": [
            {
              "name": "ModusTextInput",
              "props": { "label": "Company Name" },
              "children": []
            },
            {
              "name": "ModusTextInput",
              "props": { "label": "EIN" },
              "children": []
            }
          ]
        }
      ]
    }
  ]
}
```

The hierarchy captures:

- **Nesting** -- which components contain which, reflecting the designer's composition
- **Modus components** -- every Modus Web Component used, in context
- **Custom components** -- the designer's own components (steps, layouts) as structural nodes
- **Props** -- labels, text content, placeholders, and other meaningful prop values as key-value records

Interactive Element Extraction

The agent identifies interactive elements within the hierarchy:

1. Modus input components (`ModusTextInput`, `ModusSelect`, `ModusCheckbox`, etc.).
 2. Navigation elements (buttons, links, router `<Link>` components).
 3. For each element, the agent records `type`, `label` (visible text or placeholder), and `position` (description of location in the UI).
-

6. ADO Integration

The pipeline creates Azure DevOps work items for each ingested prototype.

Configuration

v2 change from v1: The v1 spec proposed Azure service principal OIDC authentication. The implementation uses a **Personal Access Token (PAT)** stored as `AZDO_PAT`. The v1 spec also omitted `ADO_ITERATION_PATH`, which is now included.

All ADO settings are defined in the workflow YAML `env` block and repository secrets:

Setting	Source	Value	Description
<code>ADO_ORG_URL</code>	Workflow env	<code>https://dev.azure.com/ViewpointVSO</code>	ADO organization URL
<code>ADO_PROJECT</code>	Workflow env	<code>Lista</code>	ADO project name
<code>ADO_AREA_PATH</code>	Workflow env	<code>Lista</code>	Area path for created work items
<code>ADO_ITERATION_PATH</code>	Workflow env	<code>Lista</code>	Iteration path for work items
<code>ADO_WORK_ITEM_TYPE</code>	Workflow env	<code>User Story</code>	Work item type to create

Setting	Source	Value	Description
AZDO_PAT	Actions secret	--	Personal access token for ADO REST API

Authentication

The pipeline authenticates to ADO using a **Personal Access Token (PAT)** stored as the `AZDO_PAT` repository secret. The PAT is passed to `create-ado-workitems.js` via the `AZDO_PAT` environment variable.

Composition

v2 addition: A dedicated "Compose Work Item Input" bash step merges data from three sources before calling the ADO script.

Before creating work items, an inline bash step assembles a JSON object per slug by combining:

- Validated metadata from `metadata/{slug}.json`
- Screenshot file paths from `screenshots/{slug}*.png`
- Resolved route from Phase 2
- Source path (derived from slug: folder path for folder-based pages, PascalCase `.tsx` path for single-file pages)

Tagging

All pipeline-created work items are tagged with two tags:

- `proto-ingested` -- distinguishes automated work items from manually created ones and enables board filtering.
- `slug:<slug-name>` -- enables WIQL queries for idempotency and version detection (e.g., `slug:payroll-onboarding`).

v2 change from v1: The v1 spec only mentioned `proto-ingested`. The implementation adds a `slug:` tag to enable precise WIQL filtering per prototype.

Versioning

Before creating a work item, the pipeline queries ADO for existing work items matching the `slug:<slug-name>` tag. If matches are found, the version increments (v1, v2, v3...). The new work item:

- Includes the version in the title for v2+ (e.g., "Prototype: Payroll Setup (v2)"). v1 items omit the version suffix.
- Links to the previous version via an ADO "Related" relation.

Idempotency

The commit SHA that triggered the ingestion is embedded in the work item description HTML as a `data-commit-sha` attribute. Before creating a work item, the pipeline queries ADO via WIQL for existing items matching the slug tag and checks the description for the commit SHA. If a match is found, the prototype is skipped to prevent duplicates on workflow re-runs.

v2 change from v1: The v1 spec mentioned storing the commit SHA in a "custom field or description metadata." The implementation uses a `data-commit-sha` HTML attribute in the description, queryable via WIQL and the ADO REST API.

WIQL queries use `escapeWiql()` to prevent injection via slug values containing single quotes.

Work Item Content

The ADO work item is composed with the following structure:

Title: `Prototype: <name> (v1)` or `Prototype: <name> (v<N>)` (v2+)

Tags: `proto-ingested; slug:<slug-name>`

Description (rendered as HTML):

1. Prototype name header
2. Source link (GitHub URL to source file/folder at the triggering commit)
3. Commit SHA (embedded as `data-commit-sha` attribute)
4. Route path (e.g., `/payroll/setup`)
5. Component hierarchy (recursive nested `<ul>` with component names and props)
6. Interactive elements (list with type, label, and position)
7. Screenshots (links to ADO attachment URLs)
8. Previous versions (link to prior ADO work item, when version > 1)

Attachments: Screenshot PNG files uploaded as ADO attachments and linked to the work item.

7. GitHub Action Workflow

The workflow lives at

`lista-payroll-design/.github/workflows/prototype-ingestion.yml`.

Trigger

```
None
on:
  push:
    branches: [main]
    paths:
      - "src/pages/**"
  workflow_dispatch:
    inputs:
      slugs:
        description: "Comma-separated prototype slugs to process (e.g.
employees,payroll-onboarding). Leave empty to auto-detect from last commit."
        required: false
        default: ""
```

v2 change from v1: The v1 spec only included the push trigger. The implementation adds `workflow_dispatch` for manual re-runs and targeted processing.

Concurrency

```
None
concurrency:
  group: prototype-ingestion-${{ github.ref }}
  cancel-in-progress: true
```

If two pushes land in quick succession, the earlier run is cancelled in favor of the newer one.

v2 addition: Concurrency control was not described in the v1 spec.

Runner

`ubuntu-latest` with Node.js 20. The Checkout step uses `fetch-depth: 2` so that `git diff HEAD~1 HEAD` has sufficient commit history.

Phases

The workflow is organized into five phases:

None

Phase 1: CI Checks

- npm ci
- npm run lint:all
- npm test

Phase 2: Change Detection & Route Resolution

- detect-changes.js (or manual slugs from workflow_dispatch)
- resolve-routes.js

Phase 3: Agentic Extraction (skipped if no changes)

- Install Playwright Chromium
- Install @playwright/mcp and @github/copilot globally
- Start Vite dev server (background, health-check up to 30s)
- capture-screenshots.js -> Copilot + Playwright MCP per slug
- Upload screenshots artifact
- extract-metadata.js -> Copilot per slug with validation + retry

Phase 4: ADO Integration (skipped if no metadata)

- Compose work item input (merge metadata + screenshots + routes)
- create-ado-workitems.js

Phase 5: Workflow Summary (always runs)

- generate-summary.js -> GITHUB_STEP_SUMMARY

Flow Diagram

None

flowchart TD

```
A[Push to main / workflow_dispatch] --> B{Files changed in\nsrc/pages/?}
B -->|No| C[Skip -- post summary]
B -->|Yes| D[detect-changes.js:\nGroup into prototype slugs]
D --> E[resolve-routes.js:\nParse App.tsx for routes]
E --> F[Install Playwright +\nCopilot CLI + start Vite]
F --> G[For each slug:\nCopilot + Playwright MCP\ncaptures screenshots]
G --> H[Upload screenshots\nas Actions artifact]
H --> I[For each slug:\nCopilot reads source files\nextracts metadata]
I --> J[validate-metadata.js:\nSanitize + parse + Zod validate]
J -->|Invalid| K[Retry with error\nfeedback up to 2x]
```

```

K --> J
J -->|Valid| L[metadata/slug.json]
L --> M[Compose work item input:\nmerge metadata + screenshots + routes]
M --> N{ADO work item exists\nfor slug + commit SHA?}
N -->|Yes| O[Skip -- already ingested]
N -->|No| P[Query ADO for\nexisting versions]
P --> Q[Upload screenshots\nas ADO attachments]
Q --> R[Create ADO User Story\nwith metadata + attachments]
R --> S[Link to previous\nversion if v2+]
S --> T[Collect result]
O --> T
T --> U[generate-summary.js:\nPost to GITHUB_STEP_SUMMARY]

```

v2 change from v1: The flow now shows the distinct validation/retry loop for metadata, the separate screenshot upload step (both as Actions artifact and ADO attachments), and the composition step that merges data sources before ADO creation.

Error Handling

If a prototype fails at any step -- unresolvable route, route returning a 404, ADO API error, Vite server failure, metadata validation exhausting retries -- the pipeline:

1. Logs the error with the prototype slug and failure reason.
2. Skips that prototype.
3. Continues processing remaining prototypes.
4. Tracks errors in a structured `errors` array.
5. Reports all errors in the workflow summary.

The workflow itself does not fail unless a critical infrastructure error occurs (e.g., the dev server won't start, cannot authenticate to ADO at all).

Workflow Summary

The `generate-summary.js` script runs with `if: always()` and writes a markdown summary to `GITHUB_STEP_SUMMARY` containing:

- Total prototypes detected as changed
- Counts: created, skipped (duplicate), errors
- Per-prototype table with status, version number, and ADO link
- Error details section (if any failures occurred)

8. Scripts and Prompt Templates

v2 addition: This section was not in the v1 spec. It documents the modular script architecture.

Scripts

All scripts live in `.github/scripts/` as ESM modules (Node.js 20+). Each script has corresponding tests in `.github/scripts/__tests__/`.

Script	Purpose	Inputs	Outputs
<code>detect-changes.js</code>	Find changed prototype pages via git diff	Optional base/head refs	<code>has_changes</code> , <code>slugs</code> to <code>GITHUB_OUTPUT</code>
<code>resolve-routes.js</code>	Map slugs to routes by parsing <code>App.tsx</code>	JSON array of slugs	JSON slug-to-route map to <code>GITHUB_OUTPUT</code>
<code>capture-screenshots.js</code>	Generate Copilot prompts for screenshot capture	JSON slug-to-route map	JSON array of prompts to <code>GITHUB_OUTPUT</code>
<code>extract-metadata.js</code>	Generate Copilot prompts for metadata extraction	JSON array of slugs	JSON array of prompts to <code>GITHUB_OUTPUT</code>
<code>validate-metadata.js</code>	Sanitize and validate agent output against Zod schema	Raw agent output	Validated JSON or structured error
<code>run-metadata-extraction.js</code>	Orchestrate metadata extraction with retries	JSON array of prompts	<code>metadata/{slug}.json</code> files
<code>create-ado-workitems.js</code>	Create/update ADO work items with metadata and screenshots	JSON object of prototypes	<code>workitems</code> , <code>errors</code> to <code>GITHUB_OUTPUT</code>

Script	Purpose	Inputs	Outputs
<code>generate-summary.js</code>	Render workflow summary markdown	Slugs, workitems, errors	Markdown to GITHUB_STEP_SUMMARY

Prompt Templates

Templates live in `.github/prompts/` and use `{{PLACEHOLDER}}` syntax for variable injection.

`capture-screenshot.prompt.md` -- instructs the Copilot agent to use Playwright MCP browser tools (`browser_navigate`, `browser_resize`, `browser_take_screenshot`, `browser_click`) to navigate prototype pages and capture screenshots. Variables: `{{SLUG}}`, `{{ROUTES_LIST}}`.

`extract-metadata.prompt.md` -- instructs the Copilot agent to read source files, follow imports, build the component tree, and output valid JSON matching the metadata schema. Variables: `{{NAME}}`, `{{SLUG}}`, `{{SOURCE_FILES}}`.

9. Route Validation

Every page in `src/pages/` must have a corresponding route in `App.tsx` for the pipeline to resolve its URL and capture screenshots. Route validation is enforced at two levels: CI (automated) and IDE (on-demand).

CI Check: `lint:routes`

A validation script at `scripts/validate-routes.js` that runs as part of the existing lint suite.

How it works:

1. Reads `src/App.tsx` and extracts all `import` statements that pull from `./pages/...`, mapping each import to its file path.
2. Extracts all `<Route>` elements and their `path` props, mapping each routed component to its URL path.
3. Scans `src/pages/` for all `.tsx` files and directories to build the set of all pages.
4. Compares: for each page file/folder, checks if there's a route that imports from it.
5. Reports unmatched pages as errors and exits with code 1 to fail CI.

Integration:

- Added to `package.json` as `"lint:routes": "node scripts/validate-routes.js"`
- Added to the existing `lint:all` command so it runs alongside `lint:colors`, `lint:icons`, `lint:semantic`, and other checks
- The ingestion workflow itself runs `npm run lint:all` in Phase 1, so route validation is enforced before any ingestion work begins

Example output (when validation fails):

None

Route validation failed:

```
src/pages/HomePage.tsx -- no route found in App.tsx
src/pages/payroll-settings/ -- no route found in App.tsx
```

2 pages without routes. These will skip screenshot capture in the ingestion pipeline.

Add routes in `src/App.tsx` or remove the unused pages.

IDE Skill: Route Validation

A Cursor/VS Code agent skill at

`.cursor/skills/validate-prototype-routes/SKILL.md` in the `lista-payroll-design` repo provides on-demand route validation and troubleshooting.

Capabilities:

- **Validate:** Given a page file or folder, checks that a corresponding route exists in `App.tsx`. Runs the same logic as `lint:routes` but scoped to specific files.
 - **Preview:** Shows what the pipeline would extract for a given page -- the derived slug, resolved route, component hierarchy, and interactive elements -- without triggering the actual pipeline.
 - **Troubleshoot:** Helps diagnose why a prototype's screenshot failed by checking route resolution, verifying the dev server can serve the route, and confirming the page renders.
-

10. Cursor Rules for Page Structure

Cursor rules in `.cursor/rules/` guide designers toward consistent page structure that the pipeline can reliably process. These rules are always-applied so they activate automatically when working in `src/pages/`.

Rule: `page-structure-convention`

File: `.cursor/rules/page-structure-convention.mdc`

Enforces:

- **Single-file pages** are used for simple screens with no sub-components. Named with a `Page` suffix in PascalCase (e.g., `DashboardPage.tsx`, `CustomersPage.tsx`).
- **Folder-based pages** are used when a screen has sub-components, steps, or shared layout pieces. The folder is kebab-case (e.g., `payroll-onboarding/`). The main entry component lives at the folder root with a `Page` suffix (e.g., `payroll-onboarding/PayrollOnboardingPage.tsx`).
- **Every top-level page or folder** in `src/pages/` must have a corresponding route in `App.tsx`. The `lint:routes` CI check enforces this.

Rule: `wizard-structure-convention`

File: `.cursor/rules/wizard-structure-convention.mdc`

Enforces the following structure for multi-step flows:

```
None
src/pages/<wizard-name>/
  <WizardName>Page.tsx          <-- main entry component, routed in App.tsx
  steps/                       <-- individual step components
    WelcomeStep.tsx
    CompanyInfoStep.tsx
    TaxSetupStep.tsx
    ReviewStep.tsx
  components/                   <-- shared wizard components (optional)
    WizardFooter.tsx
    WizardHeader.tsx
    WizardProgressRail.tsx
  hooks/                        <-- wizard-specific hooks (optional)
    useWizardNavigation.ts
  types/                        <-- wizard-specific types (optional)
```

```
wizard.types.ts
```

Key conventions:

- Step components go in `steps/` and use a `Step` suffix (e.g., `TaxSetupStep.tsx`).
- Shared wizard chrome (headers, footers, progress indicators) goes in `components/`.
- The main page component orchestrates step navigation and renders the current step.
- All files within the wizard folder roll up to a single prototype slug for the pipeline.
Changing any file triggers re-ingestion of the entire wizard.

Rule: `page-naming-convention`

File: `.cursor/rules/page-naming-convention.mdc`

Enforces naming patterns the pipeline uses for slug and name derivation:

- Top-level files: `<Name>Page.tsx` -- the `Page` suffix is stripped for the display name (e.g., `DashboardPage.tsx` -> "Dashboard")
- Folder names: kebab-case matching the feature name (e.g., `payroll-onboarding`) -- converted to title case for the display name (e.g., "Payroll Onboarding")
- Step files: `<Name>Step.tsx` inside `steps/`
- Avoid generic names like `index.tsx` or `page.tsx` for main page components -- use the descriptive `Page` suffix

11. Environment Variables and Secrets

v2 addition: Consolidated reference for all pipeline environment variables and secrets.

Workflow-Level Variables

Set in the workflow YAML `env:` block:

Variable	Value	Purpose
<code>ADO_ORG_URL</code>	<code>https://dev.azure.com/ViewpointVS0</code>	ADO organization URL

Variable	Value	Purpose
ADO_PROJECT	Lista	ADO project name
ADO_AREA_PATH	Lista	Area path for created work items
ADO_ITERATION_PATH	Lista	Iteration path for work items
ADO_WORK_ITEM_TYPE	User Story	ADO work item type to create

Step-Level Variables

Variable	Source	Purpose
COMMIT_SHA	<code>\${{ github.sha }}</code>	Git commit SHA for idempotency
REPO_URL	<code>\${{ github.server_url }}/\${{ github.repository }}</code>	Repository URL for source links
COPILOT_GITHUB_TOKEN	<code>\${{ secrets.COPILOT_GITHUB_TOKEN }}</code>	Auth for Copilot CLI
METADATA_MAX_RETRIES	"2"	Max retry attempts for metadata extraction
METADATA_OUTPUT_DIR	"metadata"	Directory for validated metadata JSON

Secrets

Secret	Purpose
AZDO_PAT	Personal access token for ADO REST API
COPILOT_GITHUB_TOKEN	Authentication token for GitHub Copilot CLI
